2026

# Secure Code Inspector

CBRS 503

Georges Farhat

# Table of Contents


1. General Information & Findings ..... 2
2. Prompt Log ..... 3
3. Analysis ..... 7
   3.1. Analysis without chunking ..... 7
      3.1.1. routes\b2bOrder.ts: ..... 7
      3.1.2. routes\captcha.ts: ..... 7
      3.1.3. routes\chatbot.ts: ..... 7
      3.1.4. routes\chatbot.ts: ..... 8
      3.1.5. routes\fileUpload.ts: ..... 8
      3.1.6. routes\login.ts ..... 9
      3.1.7. routes\redirect.ts ..... 9
      3.1.8. routes\showProductReviews.ts ..... 9
      3.1.9. routes\updateUserProfile.ts ..... 10
      3.1.10. routes\userProfile.ts ..... 10
      3.1.11. routes\videoHandler.ts ..... 11
   3.2. Analysis with chunking ..... 11
      3.2.1. routes\b2bOrder.ts ..... 11
      3.2.2. routes\captcha.ts ..... 12
      3.2.3. routes\chatbot.ts ..... 13
      3.2.4. routes\fileServer.ts ..... 17
      3.2.5. routes\fileUpload.ts ..... 18
      3.2.6. routes\login.ts ..... 18
      3.2.7. routes\redirect.ts ..... 20
      3.2.8. routes\showProductReviews.ts ..... 20
      3.2.9. routes\updateUserProfile.ts ..... 21
      3.2.10. routes\userProfile.ts ..... 21
      3.2.11. routes\videoHandler.ts ..... 22

# 1. General Information & Findings

The chosen scope was 11 files from the OWASP Juice Shop Github.

The chosen baseline was semgrep.

The baseline includes 18 different findings, but some are duplicates. Once culled, the vulnerability count goes down to 12 unique vulnerability or risk.

I analyzed all the files in the "routes" directory of Juice Shop using semgrep, then discarded the file in which no vulnerability was found by semgrep, and from the remainder I chose the ones with different enough vulnerabilities and OWASP Top 10 classifications.

The reports used in my analysis/comparison are saved in the "reports/" directory.

The written python script has the option to do analysis with or without chunking, and here are the different findings, when it comes to the LLM's findings only:

| | Without Chunking | With Chunking |
|---|---|---|
| Finding is real | 14 | 26 |
| Finding is false | 4 | 22 |
| Total Findings | 18 | 48 |
| Precision | 77.7% | 54.2% |

Without chunking, the LLM appears to give more accurate findings, as it has the entire file and can see the "bigger picture".

With chunking, the LLM sometimes attempts to guess what it cannot see, and as the prompt is ran more times with more inputs, it's as if it assumes a vulnerability must exist in every chunk and hallucinates them, and makes guesses, such as labeling an import with the name of "insecurity" as inherently vulnerable due to its name, even though it cannot see its contents. This results in a much larger number of findings, and less precision.

Now in comparison to the semgrep baseline:

| | Without Chunking | With Chunking |
|---|---|---|
| Exact Match | 5 | 5 |
| Correct Risk, Wrong Class | 3 | 1 |
| Close Match | 3 | 4 |
| No Match | 1 | 2 |

Exact match: the LLM found the same risk as the baseline, and tagged it with the correct OWASP Top 10 2025 category.

Correct Risk, Wrong Class: the LLM found the same risk as the baseline, but tagged it with an incorrect OWASP Top 10 2025 category

Close Match: the LLM vaguely described the same risk found by baseline, but was not exact or detailed enough.

No Match: no finding by the LLM matches or is similar to the finding in the baseline.

| | Without Chunking | With Chunking |
|---|---|---|
| Exact Match | 41.6% | |
| Partial Match | 50% | 41.6% |
| No Match | 8.3% | 16.6% |

Partial match includes both close matches, and correct risk wrong class.

## 2. Prompt Log

Prompt 1:

This is the prompt that takes code as its input at first, be it an entire file or a chunk, and attempts to scan it for vulnerabilities and give a structured output.

The original version of the prompt did not have the OWASP Top 10 2025 taxonomy as a reference, and even when explicitly instructed to use 2025 categories, the LLM defaulted to classifying its findings using OWASP Top 10 2021 taxonomy.

With the reference, the LLM started correctly using 2025 classification, but the outputs were simply the category code, for example “A05:2025”, without the name of the category. A processing function was therefore made to map the code to their correct name.

The prompts also included a JSON structure for the desired output, to instruct the LLM on what to provide and for ease of parsing.

```
"""You are an AppSec reviewer. Your task is to review the provided code, using the following
OWASP Top 10 (2025) taxonomy only:

    A01:2025 - Broken Access Control

    A02:2025 - Security Misconfiguration

    A03:2025 - Software Supply Chain Failures

    A04:2025 - Cryptographic Failures

    A05:2025 - Injection

    A06:2025 - Insecure Design

    A07:2025 - Authentication Failures

    A08:2025 - Software or Data Integrity Failures

    A09:2025 - Security Logging and Alerting Failures

    A10:2025 - Mishandling of Exceptional Conditions

    Do not hallucinate file names or lines

    and for all vulnerabilities or risks found, provide the following:

  {{

    "file": "{file_path}",

    "line": "from the input, extract the vulnerable code and put it here, and which line
number it was extracted from",

    "owasp_category": "which OWASP Top 10:2025 category the finding belongs to
here",

    "risk_summary": "risk summary of 2 or 3 lines here",

    "fix_recommendation": "specific recommended fix for the issue here",

    "confidence_score": confidence score between 0 and 1 here,

  }}"""
```

Prompt 2:

This prompt is only used when chunking is used, to deduplicate the results and consolidate similar findings in the same file.

Initially, the LLM was instructed to remove duplicated based on if two findings had identical “file” keys and similar to identical “risk_summary”, but the prompt seemed to increase the quantity of findings instead, as the LLM was most likely disregarding instructions and noticing different “line” keys and deciding that the vulnerabilities were unique.

To fix the issue, the LLM had to be given detailed and explicit instructions on what to consider and duplicate and how to deal with one. To avoid the AI’s aversion to discarding data and “losing” it, it was told to look at entries bearing the same “file” key, and group them by similar “risk_summaries”, then merge them by concatenating the “line” keys (the issue that was preventing the LLM from trimming the chunk findings). It was also given an example on what to consider separate entries, in essence entries that have different categories, or in the event they share a category, their CWE is different.

"""Role:

You are a Security Report Optimizer. Your goal is to consolidate redundant findings into a single representative entry per vulnerability type per file.

Primary Key for Deduplication:

A finding is considered a "Duplicate" if it shares the same [file] AND describes the same [Specific Vulnerability Root Cause] (e.g., Hardcoded Credentials).

Instructions:

1. **Group by Root Cause**: Scan the "risk_summary". If multiple entries in the same file describe "Hardcoded credentials," they must be merged.

2. **Merge the "line" field**: Concatenate the code snippets or line numbers into a single string, separated by a semicolon or " | ".

3. **Score Calculation**: For the "confidence_score", do not average them. Simply use the highest score from the original group of findings.

4. **Consistency**: Use the most detailed "risk_summary" and "fix_recommendation" from the group for the final consolidated entry.

5. **Format**: Your output must be a valid JSON array of objects that strictly follow the Pydantic schema provided.

### Examples of Non-Duplicates (KEEP SEPARATE)

- File A: SQL Injection (OWASP A03)

- File A: OS Command Injection (OWASP A03)

*Even though they share a category, they are different root causes. Keep both.*"""

# 3. Analysis

## 3.1. Analysis without chunking

### 3.1.1. routes\b2bOrder.ts:

A05:2025 – Injection

**Risk Summary:** The code is vulnerable to code injection attacks. An attacker could inject malicious code, potentially leading to remote code execution. The use of the 'safeEval' function does not guarantee safety, as it can still be exploited.

Verdict: Finding is real

### 3.1.2. routes\captcha.ts:

A05:2025 - Injection

**Risk Summary:** The use of eval() function can lead to code injection attacks. An attacker could potentially inject malicious code, leading to security vulnerabilities.

Verdict: Finding is real

### 3.1.3. routes\chatbot.ts:

A07:2025 - Authentication Failures

**Risk Summary:** The code uses a potentially insecure method to verify the JWT token. The token is retrieved from the request cookies or headers, but it is not validated properly. This could lead to authentication bypass or other security issues.

Verdict: Finding is false (Token is verified further down in the file)

### 3.1.4. routes\chatbot.ts:

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to a path traversal attack, allowing an attacker to access files outside the intended directory. This could lead to sensitive data exposure or code execution.

Verdict: Finding is real

### 3.1.5. routes\fileUpload.ts:

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to arbitrary file write, which can lead to code execution. An attacker can upload a zip file with a malicious file that can be written to any location on the server.

Verdict: Finding is real

A04:2025 - Cryptographic Failures

**Risk Summary:** The code is vulnerable to XML External Entity (XXE) attacks. An attacker can upload a malicious XML file that can read sensitive data from the server.

Verdict: Finding is real

A04:2025 - Cryptographic Failures

**Risk Summary:** The code is vulnerable to YAML deserialization attacks. An attacker can upload a malicious YAML file that can execute arbitrary code on the server.

Verdict: Finding is real

A06:2025 - Insecure Design

**Risk Summary:** The code does not properly validate the file type and extension. An attacker can upload a malicious file with a different extension.

Verdict: Finding is false

### 3.1.6. routes\login.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to SQL injection attacks. An attacker can inject malicious SQL code by manipulating the email or password fields.

Verdict: Finding is real

### 3.1.7. routes\redirect.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to an open redirect attack. An attacker can manipulate the 'toUrl' parameter to redirect users to a malicious website. This can lead to phishing attacks or other malicious activities.

Verdict: Finding is real

### 3.1.8. routes\showProductReviews.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to NoSQL injection attacks. An attacker could inject malicious NoSQL code, potentially leading to data breaches or unauthorized access.

Verdict: Finding is real

### 3.1.9. routes\updateUserProfile.ts

A07:2025 - Authentication Failures

**Risk Summary:** The code uses a token stored in a cookie to authenticate users. This approach is vulnerable to token theft and tampering. An attacker could steal the token and use it to access the user's account.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code checks for a specific origin or referer header to determine if the request is legitimate. However, this approach is vulnerable to header injection attacks. An attacker could manipulate the headers to bypass the check.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code updates the user's username based on the request body. This approach is vulnerable to injection attacks, as an attacker could manipulate the request body to inject malicious data.

Verdict: Finding is real

### 3.1.10. routes\userProfile.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to code injection via the username field. An attacker could inject malicious code, potentially leading to security breaches.

Verdict: Finding is real

### 3.1.11. routes\videoHandler.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to a Cross-Site Scripting (XSS) attack. The 'subs' variable is not properly sanitized, allowing an attacker to inject malicious code.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The code does not check for authentication or authorization before reading the 'promotionVideo.pug' file. This could lead to unauthorized access to sensitive data.

Verdict: Finding is false

A09:2025 - Security Logging and Alerting Failures

**Risk Summary:** The code does not log or alert on potential security issues, such as errors reading the subtitles file. This could lead to undetected security incidents.

Verdict: Finding is false

## 3.2. Analysis with chunking

### 3.2.1. routes\b2bOrder.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks as it uses the vm module to execute user-controlled input and directly evaluates user input. This could lead to remote code execution (RCE) and other security issues.

Verdict: Finding is real

A04:2025 - Cryptographic Failures

**Risk Summary:** The use of a hash function to generate an order number may not be cryptographically secure.

Verdict: Finding is real

A06:2025 - Insecure Design

**Risk Summary:** The use of the 'notevil' library, which is a sandboxed JavaScript evaluator, can still pose a security risk if not properly configured. An attacker could potentially exploit this to execute malicious code.

Verdict: Finding is real

### 3.2.2. routes\captcha.ts

A05:2025 - Injection

**Risk Summary:** The use of eval() function can lead to code injection attacks, potentially allowing an attacker to execute malicious code. This can result in unauthorized access, data tampering, or other security breaches.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The code is vulnerable to timing attacks because it uses a simple string comparison to verify the CAPTCHA answer. This can be exploited by an attacker to determine the correct answer.

Verdict: Finding is real

### 3.2.3. routes\chatbot.ts

A02:2025 - Security Misconfiguration

**Risk Summary:** The code uses a potentially insecure configuration file to load the chatbot training data. This could lead to security misconfiguration if the configuration file is not properly secured. An attacker could potentially manipulate the configuration file to load malicious training data.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to path traversal attacks and injection attacks. An attacker could manipulate the filename to write data to arbitrary locations on the file system or inject malicious data into the system.

Verdict: Finding is false

A04:2025 - Cryptographic Failures

**Risk Summary:** The code is vulnerable to JSON injection attacks. An attacker could manipulate the trainingSet variable to inject malicious JSON data.

Verdict: Finding is false

A06:2025 - Insecure Design

**Risk Summary:** The code is vulnerable to insecure design. The bot's configuration is not properly validated, which could lead to security issues.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks. The user input is directly used in the factory.run function without proper validation or sanitization.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks as it directly uses user input in the bot.respond function and botUtils function without proper validation or sanitization.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code directly uses the user input (req.body.query) to update the user's username. This could lead to injection attacks if the input is not properly sanitized.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The code is vulnerable to authentication failures because it does not properly validate the token. An attacker could potentially bypass authentication by providing a malicious token.

Verdict: Finding is false

A07:2025 - Authentication Failures

**Risk Summary:** The code does not handle the case where the user is null. This could lead to authentication failures if the user is not properly authenticated.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks because it directly accesses the username from the user object without proper validation. An attacker could potentially inject malicious data into the username.

Verdict: Finding is false

A07:2025 - Authentication Failures

**Risk Summary:** The code does not validate or sanitize the username before adding it to the bot. This could lead to authentication bypass or other security issues.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The code is vulnerable to authentication failures. It does not properly validate the token, which could lead to unauthorized access.

Verdict: Finding is false

A07:2025 - Authentication Failures

**Risk Summary:** The code does not handle the case where the user is null. This could lead to authentication failures and unauthorized access.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks. It directly uses the user input without proper validation or sanitization.

Verdict: Finding is real

A04:2025 - Cryptographic Failures

**Risk Summary:** The code uses a public key for JWT verification, but it does not check if the key is properly configured or if it has been revoked. This could lead to authentication bypass or other security issues.

Verdict: Finding is false

A10:2025 - Mishandling of Exceptional Conditions

**Risk Summary:** The code catches an error and then tries to recover by calling the bot.respond function again. However, this could lead to a situation where the error is not properly handled and the system becomes unstable.

Verdict: Finding is false

A10:2025 - Mishandling of Exceptional Conditions

**Risk Summary:** The code potentially leaks error messages to the user, which could reveal sensitive information about the application's internal workings. This could be used by an attacker to gain insight into the application's vulnerabilities. Error messages should be handled carefully to prevent information disclosure.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The authorization token is generated based on the updated user response. However, the security of the token generation process is not explicitly validated. This could lead to authentication failures if the token generation process is flawed.

Verdict: Finding is real

A09:2025 - Security Logging and Alerting Failures

**Risk Summary:** The code stores the authentication token and user response in a data structure, but it does not provide any logging or alerting mechanisms in case of authentication failures. This could lead to security logging and alerting failures.

Verdict: Finding is real

### 3.2.4. routes\fileServer.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to a null byte injection attack. An attacker could potentially inject a null byte into the file name, allowing them to access files outside of the intended directory. The code is also vulnerable to a null byte poisoning attack, which can lead to unauthorized access to files on the server.

Verdict: Finding is real

A06:2025 - Insecure Design

**Risk Summary:** The code only checks for two specific file types, but does not account for other potential file types that may be uploaded. This could lead to insecure file uploads. The function name 'endsWithAllowlistedFileType' implies that it should be checking for a list of allowed file types, but it only checks for two.

Verdict: Finding is false

### 3.2.5. routes\fileUpload.ts

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to XML External Entity (XXE) attacks, YAML injection attacks, directory traversal attacks, and code injection vulnerabilities.

Verdict: Finding is real

A06:2025 - Insecure Design

**Risk Summary:** The code does not validate the authenticity of the uploaded file and only checks for a specific list of allowed file types, which may not be comprehensive.

Verdict: Finding is real

A07:2025 - Authentication Failures

**Risk Summary:** The code uses the file's original name to determine its type, which can be tampered with by an attacker. This could lead to bypassing of file type restrictions.

Verdict: Finding is false

### 3.2.6. routes\login.ts

A02:2025 - Security Misconfiguration

**Risk Summary:** The code is importing a module named 'insecurity' which could indicate a potential security risk. The module may contain vulnerable code or insecure functions.

Verdict: Finding is false

A07:2025 - Authentication Failures

**Risk Summary:** The code does not properly validate user data before passing it to the afterLogin function, potentially allowing unauthorized access. This could lead to authentication bypass or other security issues. The lack of input validation makes the application vulnerable to various attacks.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to SQL injection attacks. An attacker can inject malicious SQL code by manipulating the email or password fields.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code is directly concatenating user input (email) with a domain string, which could lead to email injection attacks. An attacker could manipulate the email address to bypass security checks.

Verdict: Finding is false

A04:2025 - Cryptographic Failures

**Risk Summary:** Hardcoded credentials are used in the code, which is a significant security risk. An attacker can easily access the system using these credentials. This is a clear violation of secure coding practices.

Verdict: Finding is real

A10:2025 - Mishandling of Exceptional Conditions

**Risk Summary:** The code catches all exceptions and throws a generic error, potentially masking important error details. This can make debugging and error handling more difficult. It may also reveal too much information to users.

Verdict: Finding is false

### 3.2.7. routes\redirect.ts

A02:2025 - Security Misconfiguration

**Risk Summary:** The code imports a module named 'insecurity' which may indicate the presence of intentional security vulnerabilities. This could lead to various security issues, including data breaches and unauthorized access. The function isUnintendedRedirect does not properly validate URLs, potentially allowing unauthorized redirects. This could lead to phishing or other attacks. The function only checks if the URL does not start with any of the allowed URLs, but it does not check for other potential issues.

Verdict: Finding is real

### 3.2.8. routes\showProductReviews.ts

A10:2025 - Mishandling of Exceptional Conditions

**Risk Summary:** The sleep function can cause performance issues and potentially lead to Denial of Service (DoS) attacks. It can also lead to unresponsive user interface.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to NoSQL injection attacks. An attacker could inject malicious NoSQL code, potentially leading to data breaches or unauthorized access.

Verdict: Finding is real

### 3.2.9. routes\updateUserProfile.ts

A07:2025 - Authentication Failures

**Risk Summary:** The code uses a token from the request cookies to authenticate the user. This approach is vulnerable to token hijacking and replay attacks. An attacker could steal the token and use it to impersonate the user.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to CSRF attacks. It checks the origin and referer headers, but these can be spoofed. This allows an attacker to perform actions on behalf of the user.

Verdict: Finding is real

### 3.2.10. routes\userProfile.ts

A04:2025 - Cryptographic Failures

**Risk Summary:** The code uses the html-entities library to encode user input. However, this library may not provide sufficient protection against all types of injection attacks. The code is also vulnerable to CSP bypass, which can lead to XSS attacks.

Verdict: Finding is real

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to injection attacks as it directly assigns user input (username) to a variable without proper validation or sanitization. The eval() function can evaluate any JavaScript code, which makes it possible for an attacker to inject malicious code. The code is also vulnerable to template injection and XSS attacks.

Verdict: Finding is real

### 3.2.11. routes\videoHandler.ts

A06:2025 - Insecure Design

**Risk Summary:** The code snippet provided does not give enough information to determine the potential risks. However, the use of the Entities class could potentially lead to insecure design if not properly implemented. Insecure design can lead to various security risks, including data breaches and unauthorized access.

Verdict: Finding is false

A07:2025 - Authentication Failures

**Risk Summary:** The code does not validate the input path. This could lead to unauthorized access to files. An attacker could potentially access sensitive files by manipulating the input.

Verdict: Finding is false

A10:2025 - Mishandling of Exceptional Conditions

**Risk Summary:** The code uses a synchronous method to retrieve file statistics. This could lead to performance issues and potentially cause the server to hang if the file does not exist or is inaccessible.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code does not validate the range header. This could lead to a denial-of-service attack if an attacker sends a malicious range header.

Verdict: Finding is real

A02:2025 - Security Misconfiguration

**Risk Summary:** The code does not validate or sanitize the 'Content-Type' header, which could lead to security misconfiguration. This could allow an attacker to manipulate the response headers.

Verdict: Finding is false

A05:2025 - Injection

**Risk Summary:** The code is vulnerable to Cross-Site Scripting (XSS) attacks, template injection attacks, and path traversal attacks. An attacker could inject malicious scripts, templates, or file paths, potentially leading to unauthorized access or data theft.

Verdict: Finding is real